

Beyond Executability: Hierarchical Feedback for Cross-Framework Training-Code Migration Repair

Anonymous Authors
Anonymous Institution
anonymous@example.com

Abstract—Cross-framework migration of deep learning training code must preserve more than executability, since a translated program must reproduce forward values, gradients, and parameter updates across different runtime and automatic-differentiation systems. Existing translation and repair pipelines reduce feedback to binary tests or execution errors. A runnable candidate can therefore change training semantics without producing any signal that something is wrong. We argue that the granularity of repair feedback is itself a critical but overlooked factor in migration correctness. To this end, we introduce a hierarchical feedback framework that compares paired source and candidate executions through execution, forward-numerical, and gradient/update signals, reporting the first failing semantic layer as the diagnosis that drives targeted repair and re-verification. A common adapter interface keeps this logic independent of the target backend. We implement the framework on MindSpore and JAX target paths. Extensive experiments across both paths demonstrate that the full hierarchy verifies more migrations than execution-only feedback under a matched repair budget. It also exceeds the strongest blind-repair baseline on a controlled fault pool while using an order of magnitude fewer tokens. The same hierarchy transfers to a PyTorch-to-JAX path on which existing conversion tools fail outright.

Index Terms—cross-framework migration, training-code translation, hierarchical feedback, program repair, differential testing

I. INTRODUCTION

Large language models (LLMs) have made source-code translation increasingly practical, and recent systems improve generated programs through execution, repository context, and iterative feedback [1]–[5]. Cross-framework migration of deep learning (DL) training code raises a stricter problem. A migrated program must preserve not only control flow and API behavior, but also the tensor computations, automatic differentiation, optimizer state, and parameter updates that determine learning.

This requirement creates failures that ordinary executability checks cannot see. A translated training script may run successfully while producing different logits or losses. It may match the forward computation but propagate incorrect gradients, or compute plausible gradients while updating a different parameter set. Such failures arise across framework boundaries because runtimes differ in operator conventions, execution modes, differentiation interfaces, and optimizer semantics [6]–[8]. A successful run is therefore only the first condition of a correct training migration.

Existing systems supply repair feedback in three main ways. Translation pipelines validate against a single outcome such as

compilation, unit tests, or repository-level pass rates [1]–[3]. Repair agents explore the program until a test or execution trace exposes the fault [9]–[13]. Conversion and differential-testing tools compare a converted output or a single behavioral property against a reference [14]–[16]. Despite their differences, all three collapse distinct semantic failures onto one verdict. Execution failures, forward mismatches, and backward/update errors therefore arrive as the same signal even though each requires a different repair.

Our central observation is that training correctness has an ordered semantic structure. A candidate must first execute, then preserve the forward computation, and finally preserve gradients and parameter updates. The first failing layer in this sequence provides an actionable diagnosis: execution failures point to compatibility and lowering, forward failures point to operator semantics, and gradient/update failures point to automatic differentiation, parameter registration, or optimizer behavior.

Based on this observation, we propose a hierarchical feedback mechanism with execution, forward-numerical, and gradient/update signals. Paired source and candidate executions are normalized into a common diagnosis report, and a repair component uses the first failing layer to produce a targeted patch before re-verification. A common adapter interface separates this feedback logic from backend execution. We implement the method on `torch4ms/MindSpore`¹ and `TorchAX/JAX`.

The evaluation covers the effect of feedback granularity, the diagnostic behavior of each individual layer, comparison against repair agents, and transfer to a second backend. Under a matched repair budget, richer feedback repairs strictly more faults at every level of the hierarchy.

This paper makes three contributions:

- We formulate cross-framework training-code migration as an ordered semantic validation problem and identify the first failing layer as the key signal for diagnosis-guided repair.
- We design a backend-independent hierarchy over execution, forward-numerical, and gradient/update behavior and integrate it with translation, targeted repair, and re-verification.

¹The primary prototype repository is available at <https://gitee.com/feixiao13/ascend-torch4ms.git>.

- We implement the method on MindSpore and JAX target paths and establish its effect, mechanism, and advantage over repair baselines.

II. RELATED WORK

A. Translation with Outcome-Level Validation

Early neural code translation showed that programming languages can be translated without direct supervision from multilingual code corpora [17]. Later work added automated unit tests as validation and self-training signals [18]. Compiler representations were then introduced to narrow source–target semantic gaps [19]. Recent LLM-based systems incorporate executability representations, intermediate translation paths, and repository-level context [1]–[3]. MatchFixAgent decomposes repository-level equivalence checking across specialized agents and repairs the inequivalences it finds [20]. We use it as an external repair baseline. In each case validation is organized around a single outcome: compilation, unit-test results, or repository-level pass rates. Our work instead treats validation as an ordered set of semantic layers, so a passing outcome is no longer the unit of feedback.

B. Repair Driven by Execution Feedback

Automated program repair has long used tests or semantic constraints to guide patch generation [21]. Pre-trained language models made patch generation more fluent and enabled iterative repair from execution feedback [12]. Repository-level agents such as SWE-agent use interactive tools and test feedback to edit code iteratively [13]. Reflection and agent-based methods feed natural-language critiques, self-debugging traces, or executable code actions back into the generation loop [9]–[11]. Multi-agent systems show that role specialization, structured communication, and explicit workflows improve complex software tasks [22]–[25]. These approaches motivate our translator–verifier–fixer decomposition. They locate a fault by exploring the program until a test or trace reveals it. We instead read the fault location from the layer at which the two executions first diverge, which removes the search rather than guiding it.

C. Cross-System Comparison and Framework Bridging

Modern DL frameworks expose different execution models and optimization abstractions. PyTorch emphasizes an imperative, dynamic front end [6], whereas TensorFlow demonstrates large-scale dataflow execution [26]. Compiler infrastructures such as MLIR lower domain-specific computation through extensible intermediate representations [27]. Bridging tools address parts of the resulting compatibility problem. MindConverter targets migration from third-party frameworks to MindSpore [14]. Ivy² and torch2jax³ convert PyTorch computations to JAX, while TorchAX preserves a PyTorch-style front end over a JAX backend [8]. A separate line compares semantically related executions to expose defects.

Differential testing has found compiler and program-analysis bugs this way [15], [28], and DL-system testing has explored behavioral coverage and differential-style validation of networks [16], [29]. Both lines compare either the conversion output or a single behavioral property. We instead compare paired training executions at several ordered semantic depths, so the comparison itself yields the repair target.

III. PROBLEM FORMULATION

A. Task Formulation

Given a training program P in a source framework F_s and a target framework path F_t , the migration task is to produce a candidate $\hat{P}$ that executes under F_t while preserving the training semantics of P . Training semantics includes the quantities that determine whether learning remains equivalent in practice: compatible tensors and modules, comparable forward values and losses, comparable gradients, and parameter updates with comparable direction and magnitude. We instantiate this formulation on torch4ms/MindSpore and TorchAX/JAX target paths.

We evaluate correctness through paired executions under controlled inputs, synchronized initialization, and matched optimizer settings. Let $\mathcal{O}_t(P)$ denote the observable state at training step t : execution status, selected outputs or losses, gradients, and parameter-update statistics. A candidate is semantically aligned when $\mathcal{O}_t(\hat{P})$ remains within predefined tolerances of $\mathcal{O}_t(P)$ for the checked steps. All observables are framework-independent. The same diagnostic logic therefore compares executions produced by different target runtimes.

B. Diagnosis-Guided Repair Objective

The repair objective identifies the earliest semantic layer at which $\hat{P}$ departs from P and exposes evidence for the corresponding edit. Execution failures call for compatibility or lowering changes; forward mismatches call for operator-semantics changes; and gradient/update mismatches call for differentiation, parameter-registration, or optimizer changes. Each layer implies its own edit. The verifier preserves this structure in the feedback delivered to the repair component.

IV. HIERARCHICAL SEMANTIC FEEDBACK

A. Design Principle

Training-code correctness is evaluated as an ordered hierarchy. A migrated program must first execute under the target backend, then preserve the forward computation, and finally preserve the backward and update behavior that determines learning. The order is not a convention. Each layer is only measurable once the layer above it holds. We therefore organize validation around three semantic layers:

execution $\rightarrow$ forward numerical $\rightarrow$ gradient/update.

This ordering defines both a testing procedure and a repair policy. A deeper comparison is interpreted only after its prerequisites are available, and the first failing layer becomes the primary diagnosis. Later signals may provide supporting evidence, but they do not replace the earliest point at which the source and candidate executions diverge.

²<https://github.com/unifyai/ivy>

³<https://github.com/rdyro/torch2jax>

B. Framework Adapter Interface

The hierarchy operates on a common observation schema rather than framework-specific error formats. Each target path supplies an execution adapter that runs the candidate and exposes phase status, selected forward values, gradients, and parameter deltas in a normalized report. The adapter is the only backend-specific component. Runtime-specific details remain inside it; the layer ordering, comparison metrics, diagnosis format, and repair routing remain unchanged.

C. Execution Signal

The execution signal checks whether $\hat{P}$ can complete the required training step under the target backend, recording whether module construction, data movement, forward execution, loss computation, backward execution, and optimizer stepping each terminate without runtime failure. When a failure occurs, the verifier stores the exception type, stack location, operator or API name when available, tensor shape and dtype context, and the phase in which the failure appears. The phase is what makes the record actionable. Typical failure classes include missing operators, unsupported lowering paths, interface mismatches, device-placement errors, dtype incompatibilities, shape errors, and invalid parameter registration.

D. Forward-Numerical Signal

The forward-numerical signal checks whether the computation of $\hat{P}$ remains aligned with P once both executions complete. Under controlled inputs (x, y) and synchronized initialization, the verifier compares quantities such as logits, selected intermediate activations, and losses. The minimal scalar comparison is

$$\Delta_{\text{loss}} = \left| L(P; x, y) - L(\hat{P}; x, y) \right|. \quad (1)$$

When intermediate outputs are available, the verifier also computes output- and layer-level discrepancies and locates the earliest module whose difference exceeds tolerance. This localization matters because an end-to-end loss mismatch can originate in an early operator difference, a reduction convention, a broadcasting rule, a normalization layer, or the loss function itself. The scalar alone cannot separate these causes. The earliest exceeding module can.

E. Gradient and Update Signal

The gradient/update signal checks whether the migrated candidate preserves backward and optimizer behavior. This layer is necessary because two programs may produce similar forward losses while computing different gradients, updating different parameter subsets, or applying optimizer state differently. The verifier compares gradient norms and parameter-update statistics:

$$\Delta_{\text{grad}} = \left| \|\nabla_{\theta} L(P)\|_2 - \|\nabla_{\hat{\theta}} L(\hat{P})\|_2 \right|, \quad (2)$$

$$\Delta_{\text{param}} = \frac{\|\Delta\theta - \Delta\hat{\theta}\|_2}{\|\Delta\theta\|_2 + \epsilon}. \quad (3)$$

Beyond aggregate norms, the verifier records missing gradients, zero-gradient parameters, parameter-name mismatches, and update outliers. Norms alone conflate two failures. These fields separate them. A normal forward loss with missing gradients suggests an automatic-differentiation or parameter-registration issue, whereas aligned gradients with divergent updates suggests an optimizer-state or parameter-step issue.

F. Structured Diagnosis and Repair Routing

The verifier emits the status of each layer, the first failing layer, the supporting metrics, and a bounded set of root-cause hints. The repair component routes execution failures to compatibility edits, forward-numerical failures to operator-semantics edits, and gradient/update failures to backward or optimizer edits. This mapping is fixed rather than learned. The report therefore turns the hierarchy into an actionable interface between validation and repair instead of a collection of independent metrics.

V. CROSS-FRAMEWORK MIGRATION REPAIR SYSTEM

Figure 1 shows the primary instantiation as an iterative repair system rather than a single-pass converter. A translator produces an initial target-path candidate, the verifier compares paired source and candidate executions, and the fixer edits the candidate using the structured diagnosis. Each patch is restricted to the repair scope the diagnosis points to, so a gradient-layer diagnosis cannot trigger an unrelated rewrite elsewhere. An orchestrator controls rounds, stopping rules, and reversion of failed attempts. No component holds hidden state. The components communicate through a shared pool of files containing the source program, candidate program, diagnosis report, and patch log, so every repair action is tied to an executable artifact and explicit evidence.

The primary execution adapter uses `torch4ms`, which provides a PyTorch-style front end and dispatches operators, backward execution, and optimizer behavior to MindSpore-backed implementations. The second adapter uses TorchAX with JAX automatic differentiation and Optax updates. Both expose phase status, forward values, gradient evidence, and parameter-update statistics to the verifier.

For each checked training step, paired harnesses synchronize inputs, initialization, and optimizer settings before recording execution status, loss and optional layer-level differences, gradient-norm differences, and parameter-update relative ℓ_2 differences. A candidate is accepted only after its final version passes re-verification at every configured semantic layer.

VI. EVALUATION

A. Research Questions

We organize the evaluation around three questions that follow the paper’s main claim:

- **RQ1: Repair effect.** Does hierarchical feedback improve verified repair success under the same repair budget?
- **RQ2: Diagnostic mechanism.** Do the semantic layers expose the intended failure types and support repair across semantic layers and repair scopes?

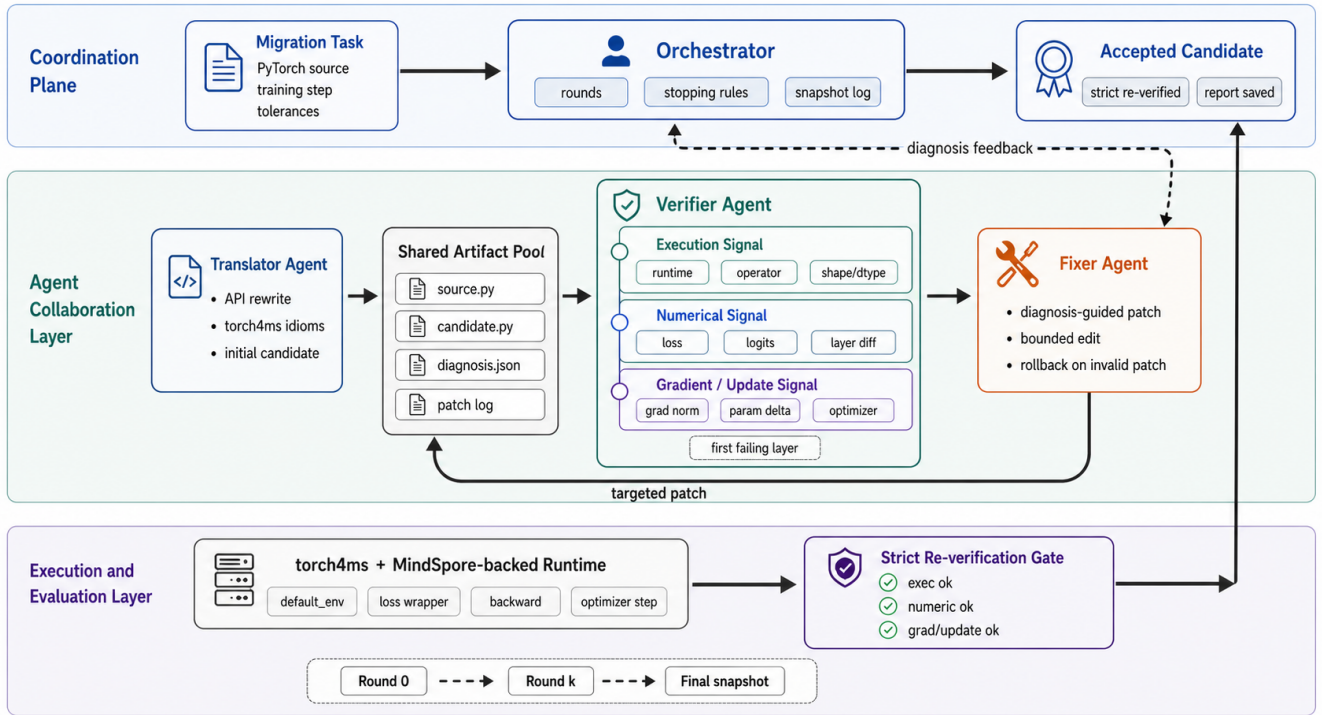


Fig. 1. The migration-repair system on `torch4ms`/MindSpore. Translation, hierarchical verification, diagnosis-guided repair, and re-verification communicate through explicit artifacts. The TorchAX/JAX implementation uses the same diagnosis schema.

- **RQ3: Comparison and transfer.** How does hierarchical feedback compare with repair and conversion baselines across MindSpore and JAX target paths?

B. Experimental Scope and Metrics

The evaluation uses vision and language training programs, controlled migration faults, and real-LLM translation and repair. The MindSpore experiments span CNN, MLP, Transformer, and tiny causal-LM families. They cover three semantic layers and three repair scopes: candidate code, core operators, and core autograd/optimizer behavior. The JAX experiment uses MLP and CNN faults at all three layers. We compare the hierarchy with two internal feedback conditions, direct LLM repair, SWE-agent, MatchFixAgent, Ivy, and `torch2jax`. The execution-only condition reports runtime status alone and withholds the numerical and gradient comparisons. The unordered condition computes every signal but presents them together as one undifferentiated set, without ranking them or naming the layer that failed first. It therefore isolates what the ordering contributes, separately from what measuring the signals contributes. Every method receives the same faulty candidate and the same repair budget. Where an external interface exposes more of the reference implementation than ours sees, we report that asymmetry with the result rather than adjusting the denominator.

The main task-level metrics are verified repair success, Repair@ k , median repair rounds, and out-of-scope edit rate where applicable. Signal-level metrics include execution status, loss and optional layer-level differences, gradient-norm

TABLE I
VERIFIED REPAIR SUCCESS UNDER MATCHED FEEDBACK CONDITIONS.

Feedback	Overall	Exec.	Numer.	Grad./upd.
Execution-only	4/12	4/4	0/4	0/4
Execution + numerical	8/12	4/4	4/4	0/4
Hierarchical (ours)	12/12	4/4	4/4	4/4

Each condition uses the same 12 tasks and four-round budget. Every successful repair finishes in one round, so Repair@1 and Repair@4 equal the displayed overall rates.

difference, and parameter-update relative ℓ_2 difference. A repair is verified only when its final artifact passes the fault-specific checks.

C. Feedback Mechanism Ablation

Design. We ran a real-LLM feedback-granularity ablation under a fixed repair budget. The ablation covers four model families: CNN, image MLP, Transformer classifier, and tiny causal LM. Each model was evaluated under exactly one fault per semantic layer: an execution fault, a numerical fault, and a gradient/update fault. Fault identifiers carry a layer prefix and an index within the 25-category pool, so the three chosen here are EX-01, NU-04, and GR-07. One fault per layer per model keeps the design balanced, which is what isolates the contribution of each layer. The repair loop used one of three feedback conditions: execution-only feedback, execution plus numerical feedback, or the full diagnosis. This produced 36 repair episodes in total. Every episode records final execution,

TABLE II
SIGNAL EFFECTIVENESS UNDER CONTROLLED FAULTS.

Model	Fault	Detected	Signals raised
CNN	Exec.	1.000	E
CNN	Num.	1.000	N+G+U
CNN	Train.	1.000	G+U
MLP	Exec.	1.000	E
MLP	Num.	1.000	N+G+U
MLP	Train.	1.000	G+U
Trans.	Exec.	1.000	E
Trans.	Num.	1.000	N+G+U
Trans.	Train.	1.000	G+U
TinyLM	Exec.	1.000	E
TinyLM	Num.	1.000	N+U
TinyLM	Train.	1.000	G+U

Detected is the fraction of seeds in which the expected signal fired, three seeds per row. E, N, G, and U denote the execution, numerical, gradient, and parameter-update signals.

numerical, and gradient/update pass fields. Verified success requires all three outcomes to pass.

Results. Table I shows a stepwise improvement as the repair loop receives the layer each fault class requires. Each added layer repairs exactly the fault class it targets and leaves the others unchanged. Execution feedback alone resolves the execution faults and none of the deeper ones. Adding numerical feedback resolves the numerical faults, and adding gradient/update feedback resolves the remainder. Every successful repair completes in the first round, so Repair@1 and Repair@4 coincide.

The layer-to-fault correspondence is exact in this controlled setting, which supports the ordering as the source of the improvement. Execution feedback handles runtime failures, forward-numerical feedback exposes runnable programs with incorrect computation, and gradient/update feedback resolves failures that remain invisible after execution and loss checks. Section VI-D2 measures the complete system over 25 fault categories.

D. Diagnostic Mechanism and Final Repair Capability

1) *Signal Localization under Controlled Faults: Design.* We evaluated whether each verifier signal detects the fault class it is designed to expose. This experiment uses controlled fault injection rather than automatic repair. For each model and seed, we compare a PyTorch reference step with a `torch4ms` candidate. Each pair runs under one of three injected conditions: an execution fault, a numerical fault, or a training fault. The evaluation uses four model families and three seeds per fault. The thresholds are $\Delta_{\text{loss}} > 0.02$, $\Delta_{\text{grad}} > 0.05$, and $\Delta_{\text{param}} > 0.03$.

Results. Table II shows that the expected signal detects its corresponding fault in all 36 fault-injection runs. Execution faults are isolated by the execution signal. For training faults, G records missing-gradient evidence and U captures the divergent parameter update. Numerical faults are detected at the forward layer even when their effects propagate to gradient

TABLE III
END-TO-END REPAIR RESULTS ON THE FIXED FAULT POOL.

(a) <i>MindSpore</i> fault pool by semantic layer			
Semantic layer	Instances	Verified	Repair@1
Execution	20	20 (100.0%)	90.0%
Numerical	14	14 (100.0%)	64.3%
Gradient/update	16	16 (100.0%)	93.8%
Overall	50	50 (100.0%)	84.0%

(b) <i>MindSpore</i> fault pool by repair scope			
Repair scope	Instances	Verified	Repair@1
Candidate-level	26	26 (100.0%)	92.3%
Core operator	20	20 (100.0%)	75.0%
Core autograd/optim.	4	4 (100.0%)	75.0%

Verified counts instances whose final artifact passes the fault-specific checks; over the full pool this equals Repair@4.

TABLE IV
REPAIR COMPARISON ON THE FIXED 50-INSTANCE FAULT POOL.

Method	Verified	R@1	R@2	R@4	Tokens
Execution-only	33/50 (66.0%)	58.0%	64.0%	66.0%	0.15M
All signals, unordered	34/50 (68.0%)	66.0%	66.0%	68.0%	0.18M
Direct LLM	40/50 (80.0%)	72.0%	76.0%	80.0%	1.58M
SWE-agent	46/50 (92.0%)	72.0%	84.0%	92.0%	7.64M
MatchFixAgent [†]	47/50 (94.0%)	74.0%	86.0%	94.0%	5.21M
Hierarchical (ours)	50/50 (100.0%)	84.0%	98.0%	100.0%	0.19M

R@k is verified success within k repair attempts. All methods use 25 fault categories with two instances per category and the same four-attempt limit. Tokens are total prompt plus completion tokens over the pool.

[†]MatchFixAgent is supplied the healthy implementation of the changed function by its fragment-pair interface, so its rate is an upper bound rather than a blind-repair result.

or update metrics. The first-failure rule therefore preserves a distinct diagnostic handle for every fault class.

2) *Final Repair Evaluation on a Fixed Fault Pool: Design.* We next evaluated whether the recorded verifier diagnoses and real-LLM fixer could be turned into repairs. This experiment used a fixed pool of 25 migration-fault categories and 50 model-assigned instances. The pool covered four model families: CNN, image MLP, Transformer classifier, and tiny causal LM. It spanned three semantic layers and three repair scopes: candidate-level code, core operators, and core autograd/optimizer behavior. All 50 instances were hidden from the repair prompts. Each instance allowed at most four repair attempts. Failed repairs were restored or rolled back.

Results. Table III shows that the final repair loop verified every instance in the pool. Coverage is complete at all three semantic layers and across all three repair scopes. Table IV compares the hierarchy with SWE-agent v1.1.0 [13], MatchFixAgent [20], and the internal execution-only, unordered, and direct-LLM conditions. All methods use the same four-attempt limit and fault-specific checks. Reporting all signals without an order improves only marginally over reporting execution status alone. Measuring the deeper layers is therefore not what helps. Ranking them and naming the layer that failed first is. MatchFixAgent is the strongest baseline, but its fragment-pair interface supplies the healthy implementation

TABLE V
VERIFIED REPAIRS ON THE PYTORCH-TO-JAX TRANSFER STUDY.

Method	Exec.	Num.	Grad/upd.	Overall
Hierarchical (ours)	2/2	2/2	2/2	6/6
Direct LLM	2/2	1/2	2/2	5/6
Ivy	0/2	0/2	0/2	0/6
torch2jax	0/2	0/2	0/2	0/6

Each semantic layer contains one MLP and one CNN fault. The hierarchy repairs all six faults in one round.

of the changed function. Its rate is therefore an upper bound rather than a blind-repair result. The hierarchy still leads it at both Repair@1 and Repair@4 while repairing blind. The token column separates the two families sharply. Both agent baselines consume more than an order of magnitude more tokens than the hierarchy. Localizing a failure by exploration is more expensive than reading it from a layered diagnosis.

E. Cross-Backend Transfer on PyTorch-to-JAX

Design. We implemented the hierarchy on a TorchAX/JAX target path [8]. This study contains MLP and CNN candidates with one execution, numerical, and gradient/update fault per model. The hierarchy and Direct LLM execute through TorchAX, JAX automatic differentiation, and Optax. For Ivy and torch2jax, we evaluate programs generated by their JAX conversion APIs. Verification compares loss, gradient norm, and parameter updates over three seeds and checks that candidate parameters use JAX arrays.

Results. Table V separates the conversion tools from the repair methods. Ivy and torch2jax produce no candidate that passes the training check at any layer. Direct LLM repair handles the execution and gradient/update faults but misses one numerical fault. The hierarchy localizes every fault at its expected first failing layer and repairs all six in one round. The hierarchy therefore transfers to a backend with a different differentiation system and optimizer stack.

F. Auxiliary Component Analyses

1) *Framework Documentation in the Fixer:* **Design.** Besides the diagnosis, the fixer prompt can carry documentation for the target framework, covering its operator conventions and optimizer interfaces. The experiment crosses pass/fail against hierarchical feedback with this documentation off against on, holding the translator fixed. It contains 24 base tasks over CNN and MLP models: 12 candidate-level tasks, eight core-operator tasks, and four core autograd/optimizer tasks. Each task ran under all four conditions with a four-round budget, producing 96 condition runs.

Results. Panel (a) of Table VI isolates the two factors. Pass/fail feedback repairs nothing, with or without the documentation. Hierarchical feedback repairs most tasks with the documentation off and somewhat fewer with it on. The feedback is therefore the dominant source of repair improvement. The documentation eliminates core-operator out-of-scope edits and lowers candidate-level success. It trades repair rate for

TABLE VI
FRAMEWORK-DOCUMENTATION ABLATIONS FOR ITERATIVE REPAIR AND ONE-SHOT TRANSLATION.

(a) Framework documentation in the fixer, by repair scope

Scope	Docs	Verified	Repair@1	Repair@4	Out-of-scope
Candidate	Off	12/12	100.0%	100.0%	0.0%
Candidate	On	8/12	58.3%	66.7%	0.0%
Core operator	Off	6/8	50.0%	75.0%	12.5%
Core operator	On	6/8	50.0%	75.0%	0.0%
Core autograd/optim.	Off	0/4	0.0%	0.0%	0.0%
Core autograd/optim.	On	0/4	0.0%	0.0%	0.0%

(b) Framework documentation in the translator, with the fixer disabled

Docs	Runs	Compile	Execute	Train	All three
Off	15	13.3%	13.3%	0.0%	0.0%
On	15	100.0%	100.0%	100.0%	100.0%

Docs indicates whether target-framework documentation was included in the prompt. Panel (a) rows all use hierarchical feedback; under pass/fail feedback every scope verified 0 tasks with the documentation either on or off, so those rows are omitted. Out-of-scope is the fraction of repairs that edited code outside the assigned scope. In panel (b), a run counts under All three when a single translation passes compilation, execution, and the training check.

scope discipline. Core autograd/optimizer repair is the hardest scope in every condition.

2) *Framework Documentation in the Translator:* **Design.** We next moved the same documentation to the translator prompt. The fixer was disabled and each run made one translation call, so no repair round could compensate for a weak first attempt. Five held-out migration tasks were evaluated: Image MLP, CNN, ResNet, Transformer classifier, and tiny causal LM. Each task ran three repeats per condition, giving 15 runs without the documentation and 15 with it.

Results. Panel (b) of Table VI shows that the documentation dominates one-shot translation. Without it, most outputs fail to compile and none complete the training check. With it, every output passes all three checks on the first attempt. Framework documentation therefore matters most before any feedback exists, whereas panel (a) shows the feedback carrying the repair loop afterwards.

G. Training-Path Validation

Design. Before interpreting any repair outcome, we check the paired execution paths against known-good oracle candidates. A disagreement here would mean the verifier reports differences that the migration did not cause. The evaluation contains CNN, NLP, Transformer, and tiny causal-LM trajectories, each run for 50 steps over three seeds. The per-step thresholds are $\Delta_{\text{loss}} \leq 0.02$, $\Delta_{\text{grad}} \leq 0.05$, and $\Delta_{\text{param}} \leq 0.03$.

Results. All four model trajectories pass the consistency check. The largest maximum loss difference is 3.815×10^{-6} , mean gradient-norm differences remain below 4×10^{-8} , and mean parameter-update relative ℓ_2 differences remain below 3.9×10^{-6} . These measurements validate the short-horizon execution paths used by the semantic verifier.

TABLE VII
ORACLE-CANDIDATE 50-STEP TRAINING CONSISTENCY.

Model	Pass	Mean ΔL	Max ΔL	Mean ΔG	Mean ΔU
CNN	1.000	4.570×10^{-7}	2.980×10^{-6}	1.510×10^{-8}	1.146×10^{-6}
NLP	1.000	6.167×10^{-7}	1.669×10^{-6}	1.441×10^{-8}	3.825×10^{-6}
Trans.	1.000	5.643×10^{-8}	2.384×10^{-7}	1.748×10^{-8}	7.913×10^{-7}
TinyLM	1.000	1.745×10^{-6}	3.815×10^{-6}	3.835×10^{-8}	1.127×10^{-6}

Each row aggregates three seeds and 50 paired steps. ΔL , ΔG , and ΔU denote loss, gradient-norm, and parameter-update differences.

H. Limitations and Threats to Validity

The MindSpore study covers four model families, 25 fault categories, three repair scopes, and 50-step training trajectories; the JAX transfer study covers six MLP/CNN faults. The results establish transfer across two target paths.

VII. CONCLUSION

We introduced a hierarchical feedback framework for cross-framework training-code migration that validates execution, forward-numerical, and gradient/update semantics in order. Specifically, the framework reports the first failing semantic layer and routes that diagnosis to a targeted repair and re-verification loop. This design makes repair guidance actionable at the layer where the divergence originates, rather than at the point of failure. A common adapter interface keeps the hierarchy independent of the target runtime and its differentiation system. Experiments across MindSpore and JAX target paths demonstrate that hierarchical feedback repairs more faults than execution-only and agent baselines at substantially lower token cost.

REFERENCES

- [1] M. He, Y. Chen, F. Yang, P. Zhao, W. Yin, Y. Kang, Q. Lin, S. Rajmohan, and D. Zhang, “ExeCoder: Empowering large language models with executability representation for code translation,” in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, Suzhou, China: Association for Computational Linguistics, 2025, pp. 7099–7125. [Online]. Available: <https://aclanthology.org/2025.emnlp-main.362/>
- [2] M. Macedo, Y. Tian, P. Nie, F. R. Cogo, and B. Adams, “InterTrans: Leveraging transitive intermediate translations to enhance LLM-based code translation,” in *Proceedings of the 47th International Conference on Software Engineering*. IEEE, 2025, pp. 2819–2831. [Online]. Available: <https://doi.org/10.1109/ICSE55347.2025.00236>
- [3] Z. Guan, X. Yin, Z. Peng, and C. Ni, “RepoTransAgent: Multi-agent LLM framework for repository-aware code translation,” *arXiv preprint arXiv:2508.17720*, 2025. [Online]. Available: <https://arxiv.org/abs/2508.17720>
- [4] E. Erer, A. Bozanta, T. Aytac, and A. Basar, “A fully automated agent for end-to-end code translation and validation,” in *Proceedings of the 2025 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, 2025, pp. 324–330. [Online]. Available: <https://doi.org/10.1109/ESEM64174.2025.00058>
- [5] H. Gray, T. Yotam, and O. Udrea, “Workflows vs agents for code translation,” *arXiv preprint arXiv:2512.14762*, 2025, neurIPS 2025 Fourth Workshop on Deep Learning for Code. [Online]. Available: <https://arxiv.org/abs/2512.14762>
- [6] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019, pp. 8024–8035. [Online]. Available: <https://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library>
- [7] MindSpore, “MindSpore: A new deep learning computing framework,” <https://www.mindspore.cn/en>, 2020, framework white paper and documentation.
- [8] Google, “torchax: A PyTorch frontend for JAX,” <https://google.github.io/torchax/>, 2025, accessed: 2026-06-02.
- [9] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Advances in Neural Information Processing Systems*, 2023. [Online]. Available: <https://arxiv.org/abs/2303.11366>
- [10] X. Chen, M. Lin, N. Schärli, and D. Zhou, “Teaching large language models to self-debug,” in *Proceedings of the Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=KuPixIqPiQ>
- [11] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and H. Ji, “Executable code actions elicit better LLM agents,” in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235. Vienna, Austria: PMLR, 2024, pp. 50 733–50 749. [Online]. Available: <https://proceedings.mlr.press/v235/wang24h.html>
- [12] C. S. Xia, Y. Wei, and L. Zhang, “Automated program repair in the era of large pre-trained language models,” in *Proceedings of the 45th International Conference on Software Engineering*. IEEE, 2023, pp. 1482–1494. [Online]. Available: <https://doi.org/10.1109/ICSE48619.2023.00129>
- [13] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, “SWE-agent: Agent-computer interfaces enable automated software engineering,” *arXiv preprint arXiv:2405.15793*, 2024. [Online]. Available: <https://arxiv.org/abs/2405.15793>
- [14] MindSpore, “Migrating from third party frameworks with MindConverter,” https://www.mindspore.cn/mindinsight/docs/en/r2.0.0-alpha/migrate_3rd_scripts_mindconverter.html, 2023, accessed: 2026-06-02.
- [15] X. Yang, Y. Chen, E. Eide, and J. Regehr, “Finding and understanding bugs in C compilers,” in *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, 2011, pp. 283–294. [Online]. Available: <https://doi.org/10.1145/1993498.1993532>
- [16] K. Pei, Y. Cao, J. Yang, and S. Jana, “DeepXplore: Automated whitebox testing of deep learning systems,” in *Proceedings of the 26th Symposium on Operating Systems Principles*. ACM, 2017, pp. 1–18. [Online]. Available: <https://doi.org/10.1145/3132747.3132785>
- [17] M.-A. Lachaux, B. Rozière, L. Chanasot, and G. Lample, “Unsupervised translation of programming languages,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 20 601–20 611. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/hash/ed23fbf18c2cd35f8c7f8de44f85c08d-Abstract.html>
- [18] B. Rozière, J. M. Zhang, F. Charton, M. Harman, G. Synnaeve, and G. Lample, “Leveraging automated unit tests for unsupervised code translation,” in *Proceedings of the Tenth International Conference on Learning Representations*, 2022. [Online]. Available: <https://openreview.net/forum?id=cmt-6KtR4c4>
- [19] M. Szafraniec, B. Rozière, H. J. Leather, P. Labatut, F. Charton, and G. Synnaeve, “Code translation with compiler representations,” in *Proceedings of the Eleventh International Conference on Learning Representations*, 2023. [Online]. Available: <https://openreview.net/forum?id=XomEU3eNeSQ>
- [20] A. R. Ibrahimzada and R. Jabbarvand, “MatchFixAgent: Language-agnostic autonomous repository-level code translation validation and repair,” 2025, arXiv:2509.16187.
- [21] H. D. T. Nguyen, D. Qi, A. Roychoudhury, and S. Chandra, “SemFix: Program repair via semantic analysis,” in *Proceedings of the 35th International Conference on Software Engineering*. IEEE, 2013, pp. 772–781. [Online]. Available: <https://doi.org/10.1109/ICSE.2013.6606623>
- [22] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, “ChatDev: Communicative agents for software development,” in *Proceedings of the*

- 62nd Annual Meeting of the Association for Computational Linguistics. Association for Computational Linguistics, 2024, pp. 15174–15186. [Online]. Available: <https://aclanthology.org/2024.acl-long.810/>
- [23] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, “MetaGPT: Meta programming for a multi-agent collaborative framework,” in *Proceedings of the Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=VtmBAGCN7o>
- [24] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” in *Proceedings of the First Conference on Language Modeling*, 2024. [Online]. Available: <https://openreview.net/forum?id=uAxxFFing2>
- [25] W. Chen, Y. Su, J. Zuo, C. Yang, C. Yuan, C.-M. Chan, H. Yu, Y. Lu, Y.-H. Hung, C. Qian, Y. Qin, X. Cong, R. Xie, Z. Liu, M. Sun, and J. Zhou, “AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors,” in *Proceedings of the Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=EHg5GDnyq1>
- [26] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard, M. Kudlur, J. Levenberg, R. Monga, S. Moore, D. G. Murray, B. Steiner, P. Tucker, V. Vasudevan, P. Warden, M. Wicke, Y. Yu, and X. Zheng, “TensorFlow: A system for large-scale machine learning,” in *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation*, 2016, pp. 265–283. [Online]. Available: <https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi>
- [27] C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shpeisman, N. Vasilache, and O. Zinenko, “MLIR: Scaling compiler infrastructure for domain specific computation,” in *Proceedings of the IEEE/ACM International Symposium on Code Generation and Optimization*, 2021, pp. 2–14. [Online]. Available: <https://doi.org/10.1109/CGO51591.2021.9370308>
- [28] V. Le, M. Afshari, and Z. Su, “Compiler validation via equivalence modulo inputs,” in *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, 2014, pp. 216–226. [Online]. Available: <https://doi.org/10.1145/2594291.2594334>
- [29] L. Ma, F. Juefei-Xu, F. Zhang, J. Sun, M. Xue, B. Li, C. Chen, T. Su, L. Li, Y. Liu, J. Zhao, and Y. Wang, “DeepGauge: Multi-granularity testing criteria for deep learning systems,” in *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*. ACM, 2018, pp. 120–131. [Online]. Available: <https://doi.org/10.1145/3238147.3238202>